Expt. No. 7

Title: *Implement a backpropagation algorithm to train a DNN with at least two hidden layers.*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 15/09/2025

Sign: _____

Experiment 7

Aim: Implement a backpropagation algorithm to train a Deep Neural Network with at least 2 hidden layers.

Software: Google Collab

Pre Lab-Questions:

Explain backpropagation algorithm for training deep neural network?

Backpropagation trains deep neural networks by propagating errors backward to update weights using gradient descent.

Post lab Questions:

Explain the role of hidden layers in back propagation?

Hidden layers in backpropagation extract intermediate features and propagate error gradients to update weights for learning complex patterns.

Theory: Backpropagation is a widely used algorithm for training feedforward neural networks. It computes the gradient of the loss function with respect to the network weights. It is very efficient, rather than naively directly computing the gradient concerning each weight. This efficiency makes it possible to use gradient methods to train multi-layer networks and update weights to minimize loss; variants such as gradient descent or stochastic gradient descent are often used.

The backpropagation algorithm works by computing the gradient of the loss function with respect to each weight via the chain rule, computing the gradient layer by layer, and iterating backward from the last layer to avoid redundant computation of intermediate terms in the chain rule.

Working: Working of Backpropagation:
Neural networks use supervised learning to generate output vectors from input vectors that the network operates on. It Compares generated output to the desired output and generates an error report if the result does not match the generated output vector. Then it adjusts the weights according to the bug report to get your desired output.

Backpropagation Algorithm:

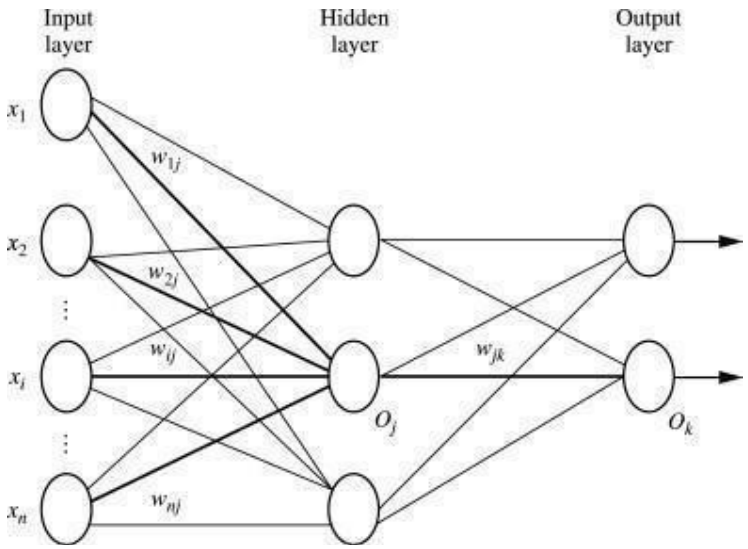
Step 1: Inputs X, arrive through the preconnected path.

Step 2: The input is modeled using true weights W. Weights are usually chosen randomly.

Step 3: Calculate the output of each neuron from the input layer to the hidden layer to the output layer.

Step 4: Calculate the error in the outputs Backpropagation Error= Actual Output – Desired Output **Step 5:** From the output layer, go back to the hidden layer to adjust the weights to reduce the error.

Step 6: Repeat the process until the desired output is achieved.



Training Algorithm:

Step 1: Initialize weight to small random values.

Step 2: While the step's stopping condition is to be false do step 3 to 10.

Step 3: For each training pair do step 4 to 9 (Feed-Forward).

Step 4: Each input unit receives the signal unit and transmits the signal x_i signal to all the units.

Step 5 : Each hidden unit Zj (z=1 to a) sums its weighted input signal to calculate its net input

$$z_{inj} = v_{0j} + \Sigma x_i v_{ij} \quad (i=1 \text{ to } n)$$

Applying activation function $z_j = f(z_{inj})$ and sends this signal's to all

Step 6: Each output unit y_k (k=1 to n) receives a target pattern corresponding to an input pattern then error is calculated as:

$$\delta_k = (t_k - y_k) + y_{ink}$$

Step 7: Each hidden unit Z_j (j=1 to a) sums its input from all units in the layer above δ_{inj}

$$= \Sigma \delta_j w_{jk}$$

The error information term is calculated as:

$$\delta_j = \delta_{inj} + z_{inj}$$

Updation of weight and bias:

Step 8: Each output unit y_k (k=1 to m) updates its bias and weight (j=1 to a). The weight correction term is given by:

$$\Delta w_{jk} = \alpha \delta_k z_j$$

and the bias correction term is given by $\Delta w_k = \alpha \delta_k$.

therefore $w_{jk(new)} = w_{jk(old)} + \Delta w_{jk}$

$$w_{0k(new)} = w_{ok(old)} + \Delta w_{ok}$$

for each hidden unit z_j (j=1 to a) update its bias and weights (i=0 to n) the weight connection term

$$\Delta v_{ij} = \alpha \delta_j x_i$$

and the bias connection on term

Step 9: Test the stopping condition. The stopping condition can be the minimization of error, number of epochs.

Program & Output

```
import numpy as np
X = np.array(([1, 9], [2, 5], [3, 6]), dtype=float)
y = np.array(([90], [85], [89]), dtype=float)

#maximum of X array longitudinally
X = X/np.amax(X,axis=0)
y = y/100

#Sigmoid Function
def sigmoid (x):
  return 1/(1 + np.exp(-x))

#Derivative of Sigmoid Function
def derivatives_sigmoid(x):
  return x * (1 - x)

#Variable initialization
epoch=2 #Setting training iterations
lr=0.1 #Setting learning rate
inputlayer_neurons = 2 #number of features in data set
hiddenlayer_neurons = 3 #number of hidden layers neurons
output_neurons = 1 #number of neurons at output layer

#weight and bias initialization
wh=np.random.uniform(size=(inputlayer_neurons,hiddenlayer_neurons))
bh=np.random.uniform(size=(1,hiddenlayer_neurons))
wout=np.random.uniform(size=(hiddenlayer_neurons,output_neurons))
bout=np.random.uniform(size=(1,output_neurons))

#draws a random range of numbers uniformly of dim x*y
for i in range(epoch):
#Forward Propogation
 hinp1=np.dot(X,wh)
 hinp=hinp1 + bh
 hlayer_act = sigmoid(hinp)
 outinp1=np.dot(hlayer_act,wout)
 outinp= outinp1+bout
 output = sigmoid(outinp)

#Backpropagation
EO = y-output
outgrad = derivatives_sigmoid(output)
d_output = EO * outgrad
```

```
EH = d_output.dot(wout.T)
hiddengrad = derivatives_sigmoid(hlayer_act)#how much hidden layer wts contributed to
error
d_hiddenlayer = EH * hiddengrad
wout += hlayer_act.T.dot(d_output) *lr # dotproduct of nextlayererror and currentlayerop
wh += X.T.dot(d_hiddenlayer) *lr

print ("----------Epoch-", i+1, "Starts----------")
print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n" ,output)
print ("----------Epoch-", i+1, "Ends----------\n")
print("Input: \n" + str(X))
print("Actual Output: \n" + str(y))
print("Predicted Output: \n" ,output)
```

OUTPUT:

```
----------Epoch- 2 Starts----------
Input:
[[0.33333333 1.        ]
 [0.66666667 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.9 ]
 [0.85]
 [0.89]]
Predicted Output:
 [[0.84923164]
 [0.84921076]
 [0.85427789]]
----------Epoch- 2 Ends----------

Input:
[[0.33333333 1.        ]
 [0.66666667 0.55555556]
 [1.         0.66666667]]
Actual Output:
[[0.9 ]
 [0.85]
 [0.89]]
Predicted Output:
 [[0.84923164]
 [0.84921076]
 [0.85427789]]
```

Conclusion:
x
Implementing the backpropagation algorithm to train a deep neural network with two hidden layers enables effective learning by adjusting weights based on error gradients. This demonstrates how deep architectures can model complex patterns through layered representations.

CO's Covered:

CO1. Implement basic neural network models.
CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.